



Christian Posta

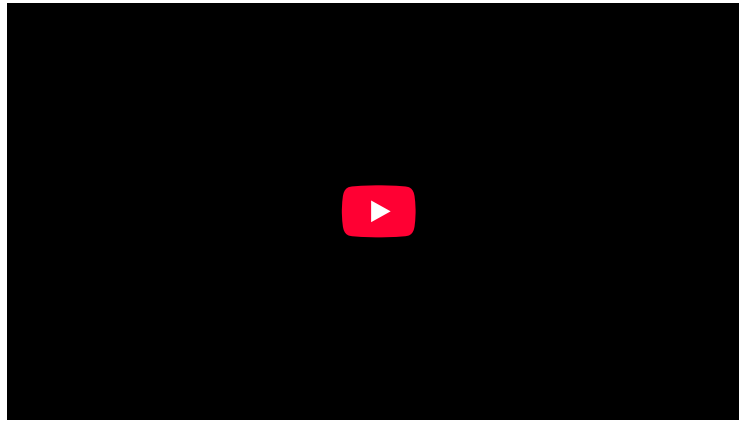
Global Field CTO at solo.io, author 'Istio in Action', 'AI Gateways in the Enterprise' and other books. He is known for being an architect, speaker, blogger and contributor to AI and infrastructure open-source projects.

LinkedIn
 Twitter
 Github
 Stackoverflow

Authenticating MCP OAuth Clients With SPIFFE and SPIRE

In the [previous blog](#), we dug into dynamically registering OAuth clients leveraging SPIFFE and SPIRE. We used SPIRE to issue software statements in the SPIFFE JWT SVID that Keycloak can trust as part of Dynamic Client Registration ([RFC 7591](#)). Once we have an OAuth client, we will want to continue to use SPIFFE to authenticate to our Authorization Server. This eliminates the need for a long-lived “client secret” which is common for [Confidential OAuth](#). This means we can use the Agent or MCP client’s identity (based on SPIFFE) for authorization flows based on OAuth. We dig into that in this blog.

TL;DR If you want to see a quick demo of this working:



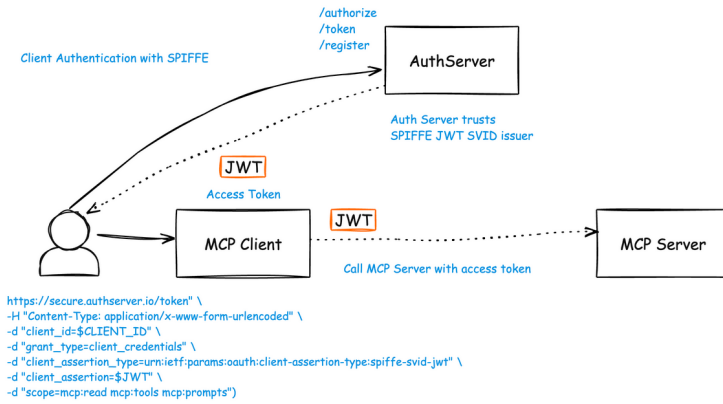
OAuth Client Authentication

[OAuth 2.0 \(and extensions like RFC 7523\)](#) specify a few ways an OAuth client can authenticate itself to the Authorization Server (AS):

- `client_secret_basic` - HTTP Basic (default)
- `client_secret_post` - Form POST
- `private_key_jwt` - JWT with private key
- `client_secret_jwt` - JWT with shared secret (less common)
- `none` - Public client (no authentication)
- `tls_client_auth` - Mutual TLS
- `self_signed_tls_client_auth` - Self-signed mutual TLS

A very common approach in microservice and machine-to-machine environments is to use a confidential client and “client credentials” flow. When the OAuth client is registered, it is issued a `client_id` and `client_secret`. This id/secret is presented to authenticate the client to the AS. The big problem with this approach is that these are usually long-lived secrets (rarely rotated) and must be kept safe somehow. Confidential clients are assumed to have some safe storage, but even so, this is an additional burden on the client to not slip up (logs, configs, copy/paste) and reveal these secrets. Lastly, these secrets are not “pre-shared secrets” and not rooted in any cryptography.

In a scenario where [SPIFFE](#) is used to issue cryptographically verifiable workload identity / agent identity / MCP client identity, we can use SPIFFE SVIDs for authenticating to the AS. That is, instead of passing static secrets, we can pass a short lived SPIFFE JWT SVIDs (or client certificates) to authenticate. An Internet Draft at the IETF has been started by Pieter Kasselmann et. al. [which describes this scenario](#). I’ve recently implemented this draft spec in some working examples I’ve been exploring and would like to share how it all works.



SPIFFE SVID Client Authentication

One question I had when digging into this is: can't we just use `private_key_jwt` (RFC 7523) to do this? That is, just give the AS the public keys for the SPIFFE/SPIRE implementation, and let the IdP/AS trust JWTs that are issued from that system?

The original intent behind `private_key_jwt` is for the OAuth client to have a private key that can be used to identify itself while the AS has the public key. So the client can create a JWT, sign it, and send it for authentication. The AS can prove that the JWT was created by the OAuth client and use that for authentication. In this scenario, Authorization Servers may expect the `iss` and `sub` claims to be the same since this is a private key scenario where the issuer should be the subject. In the SPIFFE scenario, this is not the case. Additionally, good implementations should also try to prevent replay attacks by tracking `jti`. For example, Keycloak does both of these things (checks `iss == sub` and tracks `jti`) for its implementation of RFC 7523.

Additionally, Keycloak allows setting up identity federation/brokering. The problem is, Keycloak expects a full implementation of a token provider. Using SPIRE as our SPIFFE implementation, SPIRE does not support full OAuth/OIDC token endpoints.

Since we cannot use `private_key_jwt` or identity brokering (in Keycloak), what options do we have? One option is to extend Keycloak to support a new client authentication mechanism.

Extending Keycloak for SPIFFE client authentication

To get this POC to work, we need to extend Keycloak. You can follow along in this GitHub repo to see the code.

Keycloak is written in Java and has a nice "Service Provider Interface" (SPI) model for extending many parts of Keycloak, including client authentication. To extend Keycloak to support a SPIFFE JWT authentication mechanism, we need to implement the

`ClientAuthenticatorFactory` class. I do this in the `SpiffeSvidClientAuthenticator` class:

```

public class SpiffeSvidClientAuthenticator extends AbstractClientAuthenticator {

    public static final String PROVIDER_ID = "client-spiFFE-jwt";

    @Override
    public void authenticateClient(ClientAuthenticationFlowContext context) {
        SpiffeSvidClientValidator validator = new SpiffeSvidClientValidator(context, getId());

        validator.readJwt();
        // ...more impl here...
        validator.validateToken();
        context.success();
    }

    @Override
    public Set<String> getProtocolAuthenticatorMethods(String loginProtocol) {
        if (loginProtocol.equals(OIDCLoginProtocol.LOGIN_PROTOCOL)) {
            Set<String> results = new HashSet<>();
        }
    }
}
  
```

```

    results.add("spiffe_svid_jwt");
    return results;
  }
}
}

```

A couple things to notice here. We specify a `PROVIDER_ID` of `client-spiffe-jwt` which can be used under the covers (ie, Keycloak Admin REST API) in Keycloak to refer to this configuration. We also implement an “authenticator method” `spiffe_svid_jwt` which can be used by OAuth clients in authorization flows to identify which authentication method to use (ie, `urn:ietf:params:oauth:client-assertion-type:spiffe-svid-jwt`). Not shown above, but you can check the code, we can also extend the configuration that you see in the UI to specify additional properties that can be used in the custom client authenticator. For example, I added an `issuer` property that can be configured and used in the custom client authentication validation.

From here, we need to load this into a stock Keycloak (we use a recent version at the time of writing). Here’s an example using Docker Compose:

```

services:
  keycloak-ldap:
    image: quay.io/keycloak/keycloak:26.2.5
    environment:
      KC_HEALTH_ENABLED: "true"
      KEYCLOAK_ADMIN: admin
      KEYCLOAK_ADMIN_PASSWORD: admin
    ports:
      - "8080:8080"
    volumes:
      - ./spiffe-svid-client-authenticator-1.0.0.jar:/opt/keycloak/providers/spiffe-svid-client-authenticator-1.0.0.jar
    command: start-dev
    networks:
      - keycloak-shared-network

```

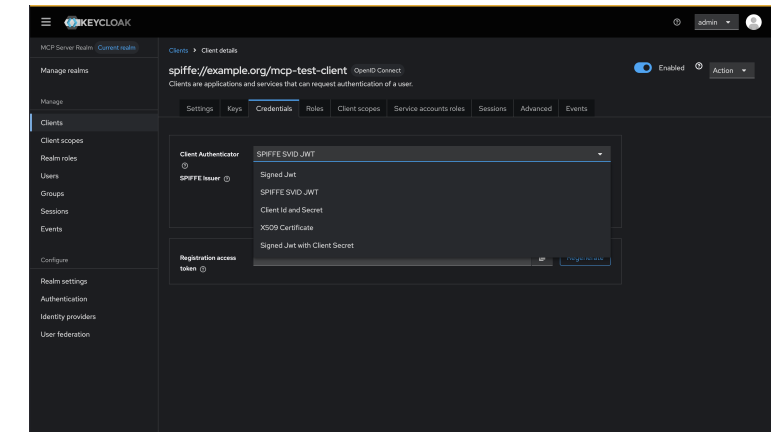
When we start Keycloak, we should see that our SPI gets loaded:

```

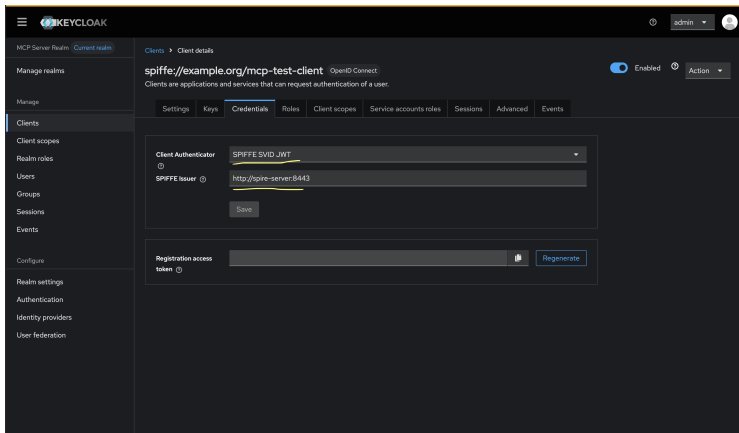
keycloak-ldap-1 | 2025-07-29 02:03:09,255 WARN [org.keycloak.services] (build-38) KC-SERVICE:
(com.yourcompany.keycloak.authenticator.SpiffeSvidClientAuthenticator) is implementing the in
This SPI is internal and may change without notice

```

If we go to an existing OAuth client (or create a new one), and navigate to the `Credentials` tab, we should see the new SPIFFE SVID JWT authenticator type.



If we select the SPIFFE SVID JWT authenticator, we can see our custom configuration fields (just one in this case, `issuer`):



We will configure the issuer with the SPIRE server address. We will also **need to configure the JWKS** that Keycloak should trust, but **SPIRE doesn't support this out of the box**. Luckily, they have a pre-built addon to support OIDC style discovery.

SPIRE OIDC Discovery Endpoint

SPIRE is a workload attestation engine and implements the SPIFFE spec. It can issue x509 or JWT SVIDs. For JWTs, it does not expose its public key/JWKS out of the box. Luckily, a simple **JWKS discovery endpoint** is available to support an OAuth federation / brokering scenario. We need to stand this up and configure it to work with our SPIRE server.

Here's an example using Docker Compose:

```
spire-oidc-discovery:
  image: ghcr.io/spiffe/oidc-discovery-provider:1.12.4
  container_name: spire-oidc-discovery
  depends_on:
    - spire-server
  ports:
    - "18443:8443"
  volumes:
    - ./oidc-discovery-provider.conf:/opt/spire/conf/oidc-discovery-provider.conf:ro
    - spire-server-socket:/tmp/spire-server/private:ro
  working_dir: /opt/spire/conf
  command: ["-config", "oidc-discovery-provider.conf"]
  networks:
    - keycloak_keycloak-shared-network
```

Note, the SPIRE OIDC discovery endpoint needs its own configuration and access to the SPIRE server. Ideally this endpoint is co-located with the SPIRE server and can access the SPIRE server's Unix Domain Socket (UDS). Here's our configuration for the OIDC discovery endpoint (note, for demo purposes, I'm using an insecure/http endpoint):

```
log_level = "INFO"
domains = ["spire-server", "spire-oidc-discovery", "localhost"]

# Use HTTP for local development (no certificates needed)
insecure_addr = ":8443"
allow_insecure_scheme = true

server_api {
  address = "unix:///tmp/spire-server/private/api.sock"
}

health_checks {}
```

Lastly, we'll need to tune some parameters on the `server.conf` for the SPIRE server itself:

```
server {
  ...
}
```

```

# Add JWT issuer for OIDC (using HTTP for local development)
jwt_issuer = "http://spire-server:8443"
default_jwt_svid_ttl = "1m"

# Configure RSA key type (required for OIDC)
ca_key_type = "rsa-2048"

# Add federation bundle endpoint
federation {
    bundle_endpoint {
        address = "0.0.0.0"
        port = 8443
    }
}
}

```

If we curl this discovery endpoint, we can see the discovery metadata and keys:

```

> curl -L http://localhost:18443/.well-known/openid-configuration
{
  "issuer": "http://localhost:18443",
  "jwks_uri": "http://localhost:18443/keys",
  "authorization_endpoint": "",
  "response_types_supported": [
    "id_token"
  ],
  "subject_types_supported": [
    "public"
  ],
  "id_token_signing_alg_values_supported": [
    "RS256",
    "ES256",
    "ES384"
  ]
}

```

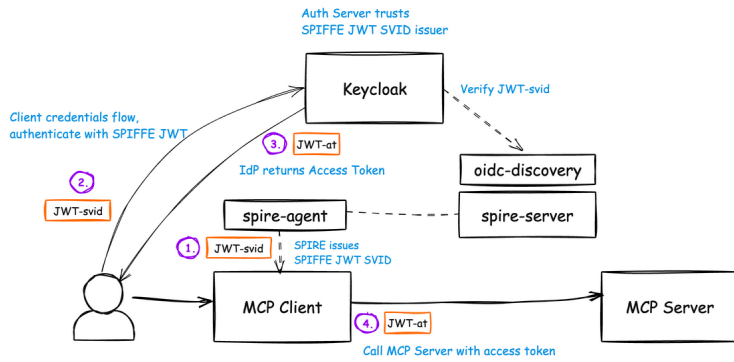
JWKS endpoint:

```

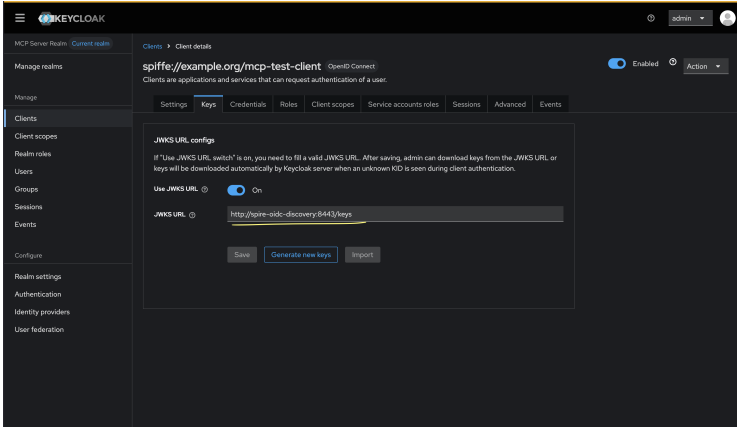
> curl -L http://localhost:18443/keys
{
  "keys": [
    {
      "kty": "RSA",
      "kid": "n0xvkL8A2W3DofkHTJPv1GpeEBJeQB6g",
      "alg": "RS256",
      "n": "sAp_Vd-X-W7O1lYPm_TTk0zvUj443Y9MfQvy4onBcursyxOajcoeSOeNpTdh4QEmLKV3xC8ZqYv4fkzFp6UTf-_rwPs_uwOpbhPKT-QQ2Kcoconxaf8RkA0m-mzOVHbU7eA3esHLTzN84kbGkrlwozQesyC-MHFE3EwLR9xI1YZfWbHt1X0cnTgBXitgysM5Yw4jkXy7kYvjs21MyEJ01_WSSHCLaISajlAvnDLWiGV3xx0Vd29m8-mrR5pg4_eicBifxnQnksO_LWRy8jXKk2JTftRKnmIxxqHML_fbVej8RSsaGpu0askj83gZ4wNDi8KNh7c9ir6yWl9jgDJ3lYQ",
      "e": "AQAB"
    }
  ]
}

```

See the [SPIRE OIDC Discovery Provider](#) for more.



With this setup, we can now configure the Keycloak JWKS endpoint to point to the SPIRE OIDC Discovery endpoint:



OAuth Client Authentication with SPIFFE in Action

With Keycloak configured to use our SPIFFE SVID JWT authenticator, and correctly pointing to the SPIRE JWKS, we can now get a workload SVID and make a call to Keycloak for an authorization flow / client credentials flow to get an access token. To get a SPIFFE JWT SVID, we can call the `spire-agent` workload API. Here's an example SPIFFE JWT SVID:

```

{
  "aud": [
    "http://localhost:8080/realms/mcp-realm"
  ],
  "client_auth": "client-spiFFE-jwt",
  "environment": "production",
  "exp": 1753800643,
  "iat": 1753800583,
  "iss": "http://spire-server:8443",
  "jwks_url": "http://spire-oidc-discovery:8443/keys",
  "organization": "Solo.io Agent IAM",
  "scope": "mcp:read mcp:tools mcp:prompts",
  "sub": "spiffe://example.org/mcp-test-client"
}

```

This JWT is signed by spiffe with the correct SPIFFE ID (`spiffe://example.org/mcp-test-client`). It has a tight expiration period, and it has additional software statements. Note the `client_auth` software statement / claim here points to `client-spiFFE-jwt` which was the `PROVIDER_ID` we specified in our `SpiffeSvidClientAuthenticator` class.

With this SPIFFE JWT SVID, we can call the token endpoint with the `spiffe-svid-jwt` and SJWT client assertions. In this particular example, we are using a `client_credentials` flow:

```

curl -s -X POST \
  "$KEYCLOAK_URL/realms/$KEYCLOAK_REALM/protocol/openid-connect/token" \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -d "client_id=$CLIENT_ID" \

```

```
-d "grant_type=client_credentials" \
-d "client_assertion_type=urn:ietf:params:oauth:client-assertion-type:spiffe-svid-jwt" \
-d "client_assertion=$JWT" \
-d "scope=mcp:read mcp:tools mcp:prompts"
```

If this is successful, Keycloak will issue an access token:

```
{
  "exp": 1753804189,
  "iat": 1753800589,
  "jti": "trrtcc:35d1fb20-31fa-4055-afb8-e902d0dc25d4",
  "iss": "http://localhost:8080/realms/mcp-realm",
  "sub": "6e4b5bc5-9a5c-4f87-aa1e-06ad279da0c8",
  "typ": "Bearer",
  "azp": "spiffe://example.org/mcp-test-client",
  "acr": "1",
  "scope": "profile email",
  "email_verified": false,
  "clientHost": "192.168.65.1",
  "preferred_username": "service-account-spiffe://example.org/mcp-test-client",
  "clientAddress": "192.168.65.1",
  "client_id": "spiffe://example.org/mcp-test-client"
}
```

Wrapping Up

In this post, we explored how Agent / MCP identity based on SPIFFE can be used as a first-class authentication mechanism for OAuth clients. By integrating SPIFFE JWT SVIDs with Keycloak’s client authentication flow, we eliminated the need for static secrets and created a more secure, scalable model for authenticating MCP clients especially in environments where agents and services need short-lived, verifiable credentials.

While this approach required some customization in Keycloak (through its SPI model) and configuration of the SPIRE OIDC Discovery endpoint, the end result is a working OAuth flow powered by cryptographically-verifiable, zero-trust-friendly identity. This isn’t just a more secure option, it’s a necessary evolution as we shift toward AI-native, agentic architectures that demand dynamic trust relationships and automated credential management.

SHARE ON

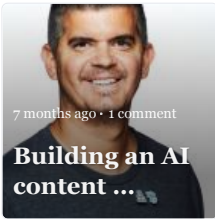
 TWITTER

 FACEBOOK

 GOOGLE+

Authenticating MCP OAuth Clients With SPIFFE and SPIRE was published on July 29, 2025.

ALSO ON CHRISTIAN POSTA SOFTWARE BLOG



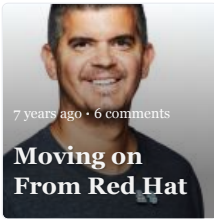
7 months ago • 1 comment

Building an AI content ...



6 years ago • 0 comments

Istio as an Example of When Not to



7 years ago • 6 comments

Moving on From Red Hat



4 months ago

Wi For Fin

Start the discussion...

LOG IN WITH

OR SIGN UP WITH DISQUS 



Name

 • Share

Best Newest Oldest

Be the first to comment.

YOU MIGHT ALSO ENJOY

[\(VIEW ALL POSTS\)](#)

- [Mitigate Prompt Injection Attacks With A2AS and Agentgateway](#)
- [Building an MCP Gateway with Apigee API Gateway](#)
- [Understanding Sessions in Agent to Agent Communication](#)